面向对象程序设计

C++新特性

★第1-4章 C++与C基础知识点差异提炼

西南交通大学
Southwest Jiaotong University

C++的新特性

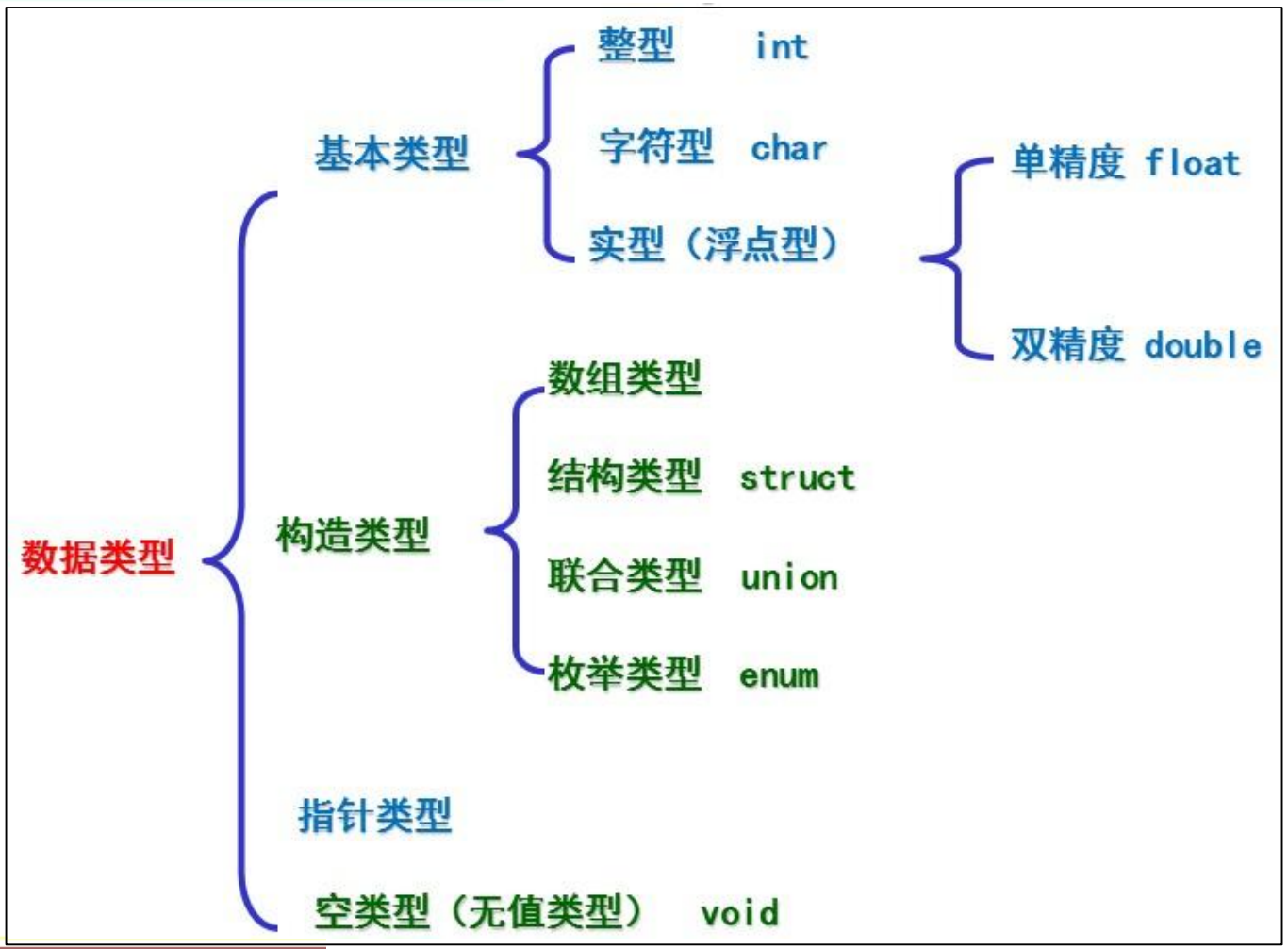
1. C++数据的输入输出 (1.4.3节)
2. 名字空间namespace (1.4.2节)
3. 引入新的数据类型(2.1.2节、10章、11章)
4. 带默认参数的函数 (2.5.3节)
5. 内联函数 (2.5.4节)
6. 函数重载 (2.5.5节)
7. 内存管理 (3.3.5节)
8. const (3.3.7节)
9. C++引用 (3.4节)
10. 异常处理 (12章)

★C++新特性：新的数据类型)(P20)

3

西南交通大学 Southwest Jiaotong University

沿用C语言中的数据类型



★C++新特性： 新的数据类型

1. bool（布尔）——C++中新的数据类型

	逻辑类型	真	假
C	没提供	非0	0
C++	**bool**	**true**	**false**

Example

```
int flag=0;
if(flag==1)
{
//to do
}
else
{
      //to do

}
```

C

```
bool flag=true ;
if(flag)
{
//to do
}
else
{
      //to do

}
```

C++

★C++新特性： 新的数据类型

2. string（字符串类型）— C++中新的数据类型（P372）

	字符串类型	表现形式
C	char str[81];	char str[81]= “Hello!” ;
		str= “Hi” ;×
C++	string	string str(“Hello!”);
		str= “Hi” ;✓

string 是一个模板类，引入#include<string>

3. string类处理字符串

使用string类必须包括头文件#include<string>

1. 构造函数常用形式

(1) string(); //默认构造函数，建立串长为0的串
(2) string(const char *s); //用C语言串s构造串
(3) string(const string &t); //用串t拷贝构造串

例：
```
string s1; cin>>s1;                    //形式(1)
string s2("abcd"), s3="123";           //形式(2)
cout<<s2<<s3<<endl;                    //输出abcd123
string s4(s2), s5=s3;                  //形式(3)
cout<<s4<<s5<<endl;                    //输出abcd123
```

3. string类处理字符串

(5) string &insert(unsigned i, string &s);
string &insert(unsigned i, char *s);
//串s插入到本串位置i之前
//i=0，插入到本串最开头
//i= 串长，插入到本串结尾，等效于append函数
例： string s("abcd");
s.insert(0, "12"); cout<<s; //输出12abcd
s.insert(s.length(), "ef"); cout<<s; //输出12abcdef
string t("ABCD"), r("XX"); t.insert(2,r);
cout<<t; //输出ABXXCD

3. string类处理字符串

(6) string substr(unsigned i, unsigned n);

//本串从位置i开始提取n个字符生成新串

例： string s("abcdef");

string t=s.substr(1,3);

cout<<t; //输出bcd

(7) int find(const string & str);

int find(const char *str);

//本串查找子串第一次出现的位置

例： string s("abcdef"); int m=s.find("bcd");

cout<<m; //输出1

3. string类处理字符串

运算符	示例	说明
+	s+t	将串s和串t连接成一个新串
=	s=t	串赋值，用t更新s
+=	s+=t	等价于s=s+t
==	s==t	判断s和t是否相等
!=	s!=t	判断s和t是否不相等
<	s<t	判断s是否小于t
<=	s<=t	判断s是否小于或等于t
>	s>t	判断s是否大于t
>=	s>=t	判断s是否大于或等于t
[]	s[i]	访问下标为i的字符(i从0开始)

3. string类处理字符串

例: string类实现串逆序存储。

```
#include <iostream>
#include <string>
using namespace std;
void main() {
  string s;  int n,i;
  cout<<“输入一个串:";
  cin>>s;
  n=s.length();
  for(i=0; i<n/2; i++) swap(s[i],s [n-i-1]);  //系统swap函数
  cout<<"s="<<s<<endl;
}
```

3. string类处理字符串

例: string类实现删除串中小写英文字符。

```
#include <iostream>
#include <string>
using namespace std;
void main() {
 string s;  int n,i,j;
 cout<<"输入一个串:";   cin>>s;
 n=s.length();
 for(i=j=0;i<n;i++) if(!(s[i]>='a'&&s[i]<='z')) s[j++]=s[i];
 s=s.substr(0,j);
 cout<<s<<endl;
 }
```

★C++新特性： 新的数据类型

西南交通大学 Southwest Jiaotong University

例：C++字符串的输入输出

更多示例：例10.6，10.7，10.8)

```
#include <iostream>
using namespace std;//使用命名空间std
int main () {
    char mybuffer [100];
    cout << "What's your name? ";
    cin.getline (mybuffer,100);
    cout << "Hello " << mybuffer << ".\n";
    cout << "Which is your favourite team? ";
    cin.getline (mybuffer,100);
    cout << "I like " << mybuffer << " too.\n";
    return 0;
}
```

运行结果：

```
What's your name? Juan
Hello Juan.
Which is your favourite team? Inter Milan
I like Inter Milan too.
```

西南交通大学 Southwest Jiaotong University

4. 数据类型的初始化

```
int a(3);                    //int a=3;
double f(3.14);
char c('a');
string str("hello");         //新的数据类型
bool t(false);               //新的数据类型
```

★C++新特性：引用&(P.124，3.4引用)

西南交通大学 Southwest Jiaotong University

- 下面的写法定义了一个**引用**，并将**其初始化为引用某个变量**。

类型名 &引 用名 = 某变量名;

引用（reference）是C + +的一种**新的变量类型**，是对C的一个重要扩充。

```
int n = 4;
int *p=&n;
int &r=n;    //r引用了n，r的类型?
```

注意：变量**r并没有**开辟新的存贮空间，它和变量**n**都是同一个变量，有**共同的存贮空间**。

- 某个变量的**引用**，**等价于**这个变量，相当于该变量的**一个别名**。

★C++新特性：引用&

西南交通大学 Southwest Jiaotong University

1. 引用的基本用法

```
int n=10;
int &r=n;
r=4;
cout<<r<<endl;//输出4
cout<<n<<endl;//输出4
n=5;
cout<<r<<endl;//输出5
```

```
double a=4,b=5;
double &r1=a;//r1引用变量a
double &r2=r1;//r2也引用变量a
r2=10;
cout<<a<<endl;//输出变量a; 10
r1=b;//r1没有引用b
cout<<a<<endl;//输出变量a; 5
cout<<endl<<endl;
```

- 定义引用时一定要将其初始化成引用某个变量。
- 初始化后，它就一直引用该变量，不会再引用别的变量了。(从一而终)
- 引用使用只针对变量，不能引用常量和表达式。

★C++新特性：引用&

执行以下语句：

```
int a=1, b=2, &p=a, &q=p;
q=b;
cout<<p<<endl<<q<<endl;
```



输出的第一行是[填空1]，输出的第二行是[填空2]。

★C++新特性：引用&

2. 指针类型的引用（指针引用）

指针类型的引用类似于二级指针（指针的指针）

类型标识符 * &引用名=目标变量名;

```
void test()
{
    int a=10,b=20;
    int *p1=&a;
    int *p2=&b;
    int *&r1=p1;
    int *&r2=p2;

    cout<<r1<<"="<<*r1<<endl;
    cout<<r2<<"="<<*r2<<endl;

    r1=r2;//(1)
    cout<<r1<<"="<<*r1<<endl;
    cout<<r2<<"="<<*r2<<endl;
}
```

//(2)替换成*r1=*r2;

① r1=r2;

0x6bfee4=10
0x6bfee0=20
0x6bfee0=20 //内存单元一致
0x6bfee0=20

② (2) *r1=*r2;

0x6bfee4=10
0x6bfee0=20
0x6bfee4=20
0x6bfee0=20

★C++新特性：引用&

3. 引用作为函数的参数

C语言中，如何编写交换两个整型变量值的函数?

```
#include<stdio.h>
int main()
{
    void swap(int *a,int *b);
    int n1=15,n2=80;
    printf("n1=%d,n2=%d\n",n1,n2);
    swap(&n1,&n2);//n1,n2的值被交换
    printf("n1=%d,n2=%d\n",n1,n2);

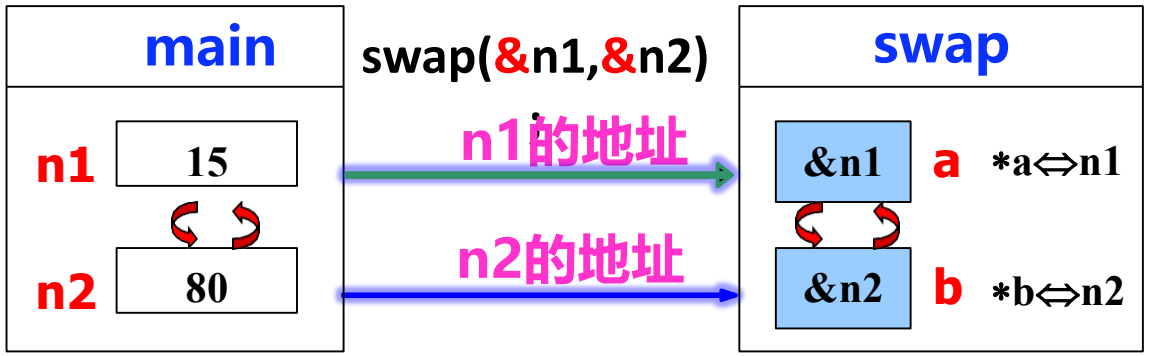
    return 0;
}
void swap(int *a,int *b)
{
    int tmp;
    tmp = *a;
    *a = *b;
    *b = tmp;
}
```

C程序

地址传递

指针形参，可以实现主调函数中两个变量换值

函数调用



输出结果：

n1=15,n2=80

n1=80,n2=15

★C++新特性：引用&

西南交通大学 Southwest Jiaotong University

□ 有了C++的引用：交换两个整型变量值的函数

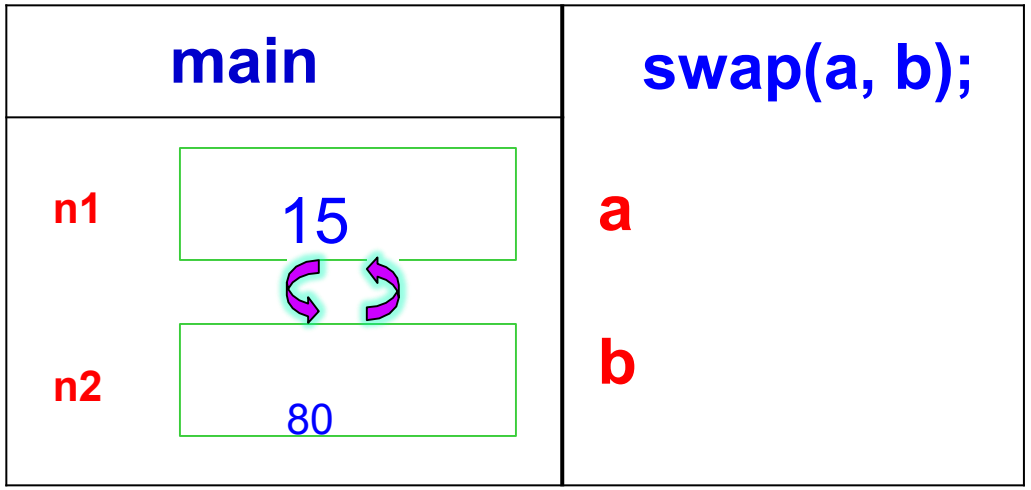
```
#include<iostream>
using namespace std;
int main()
{
    void swap(int &a,int &b);
    int n1=15,n2=80;
    cout<<"n1="<<n1<<";n2="<<n2<<endl;
    swap(n1,n2);//n1,n2的值被交换
    cout<<"n1="<<n1<<";n2="<<n2<<endl;
    return 0;
}
void swap(int &a,int &b)
{//形参是引用类型
    int tmp;
    tmp = a;
    a = b;
    b = tmp;
}
```

C++程序

引用传递

引用类似于一级指针

函数调用



引用传递实质是指针传递

书写方式像值传递

★C++新特性：引用&

3. 引用作为函数的参数

C程序

```
#include<stdio.h>
int main()
{
    void swap(int *a,int *b);
    int n1=15,n2=80;
    printf("n1=%d,n2=%d\n",n1,n2);
    swap(&n1,&n2);//n1,n2的值被交换
    printf("n1=%d,n2=%d\n",n1,n2);

    return 0;
}
void swap(int *a,int *b)
{
    int tmp;
    tmp = *a;
    *a = *b;
    *b = tmp;
}
```

地址传递

C++程序

```
#include<iostream>
using namespace std;
int main()
{
    void swap(int &a,int &b);
    int n1=15,n2=80;
    cout<<"n1="<<n1<<";n2="<<n2<<endl;
    swap(n1,n2);//n1,n2的值被交换
    cout<<"n1="<<n1<<";n2="<<n2<<endl;
    return 0;
}
void swap(int &a,int &b)
{//形参是引用类型
    int tmp;
    tmp = a;
    a = b;
    b = tmp;
}
```

引用传递

★C++新特性：引用&



若函数定义为Fun(int &i), 变量定义n=100, 则下面调用正确的是() 。

A Fun(20)

B Fun(*n)

C Fun(n)

D Fun(&n)

★C++新特性：引用&

4. 引用作为函数的返回值

满足左值定理

运行结果：40

```
#include<iostream>
using namespace std;
int n = 4;
int & SetValue()
{
    return n;
}
int main()
{
    SetValue() = 40;
    cout<<n<<endl;
    return 0;
}
```

★C++新特性：引用&

4. 引用作为函数的返回值

```
#include<iostream>
using namespace std;
int x;//全局变量
int *f(int *y)
{
    x = ++(*y);
    return &x;
}
int & g(int &y)
{
    x = ++y;
    return x;
}
```

```
int & h()
{
    int q;
    //return q;//错误
    return x;
}
int main()
{
    int a = 0;
    f(&a);
    cout<<"x="<<x<<endl;
    g(a);
    cout<<"x="<<x<<endl;
    h()=16;
    cout<<"x="<<x<<endl;
    return 0;
}
```

综合练习

```
x=1
x=2
x=16
请按任意键继续. . .
```

★C++新特性：引用&

西南交通大学 Southwest Jiaotong University

5. 引用和指针的关联

	引用	指针
1. 为空	不能为空	可以为空
2. 定义	引用跟原来的变量实质上是同一个东西，只不过是原变量的一个别名而已	指针是一个变量，只不过这个变量存储的是一个地址，指向内存的一个存储单元；
3. 可变性	引用只能在定义时被初始化一次，之后不可变	指针可变
4. 函数参数传递	□ 可以修改调用函数实参内容 □ 数据对象较大（相当于指针传递）	数组传递（必须用指针）

★引用的另类使用

课堂完成：枚举的定义，完成枚举变量c数据的输入P87

```
#include<iostream>
using namespace std;
typedef enum {
        Friend=1,Relatives=2,Classmate=3
}Relation;
int main()
{
        Relation c;
        cout<<"input:1-Friend 2-Relatives 3-Classmate"<<endl;
        // (1) 此处补充输入代码


        //输出
        cout<<"type="<<c<<endl;
        return 0;
}
```

枚举型变量不能直接用cin和cout输入输出

★复习：引用的另类使用

课堂完成：枚举的定义，完成枚举变量c数据的输入（P87）

```
#include<iostream>
using namespace std;
typedef enum {
        Friend=1,Relatives=2,Classmate=3
}Relation;
int main()
{
        Relation c;
        cout<<"input:1-Friend 2-Relatives 3-Classmate"<<endl;
        // (1) 此处补充输入代码
            cin>>(int&)c ;
        //输出
        cout<<"type="<<c<<endl;
        return 0;
}
```

枚举型变量不能直接用cin和cout输入输出

引用的左值特性，强制将枚举型转换为int型，实现赋值操作。

★枚举类型变量的输入与输出

西南交通大学 Southwest Jiaotong University

```
#include<iostream>
using namespace std;
typedef enum { Friend = 1, Relatives = 2, Classmate = 3} Relation;
int main(){
    Relation c; int input; // 先用int接收输入，避免强制转换的风险
    cout << "input:1-Friend 2-Relatives 3-Classmate" << endl;
    // 1. 接收输入并校验合法性
    if (!(cin >> input) || input < 1 || input > 3) {
        cout << "输入错误！请输入1、2、3中的整数" << endl;  return 1;
    }
    // 2. 安全的类型转换（替代强制转换）
    c = static_cast<Relation>(input);
    // 3. 输出语义化结果（而非数字）
    cout << "type=";
    switch(c) {
        case Friend: cout << "Friend"; break;
        case Relatives: cout << "Relatives"; break;
        case Classmate: cout << "Classmate"; break;
    }
    cout << endl;    return 0;
}
```

★C++新特性： const (P29)

西南交通大学 Southwest Jiaotong University

1. const与基本类型结合

const 类型 变量名=value; //定义必须初始化值

```
const double PI=3.14;
const string str="Peking University";
```

2. const与指针<常量指针与指针常量>

// 选择题：选出（1） （2） （3） （4）存在语法错误的语句。为什么?

```
int n,m;
const int *p1=&n;          //常量指针, 指针指向的内容是常量(*p1是常量)， 不可变
int * const p2=&m;         //指针常量, 指针本身是常量(p2是常量)， 不可变
*p1=5;      //  (1)
*p2=5;      //  (2)
p1=&m;      //  (3)
p2=&n;      //  (4)
```

```
const int * p = & n;
int   const * p = & n;
```

完全等价
const在*左侧

```
int * const p;    //不一样，const在*右侧
```

★C++新特性：const

常量指针用法注意事项

- 不能把常量指针赋值给非常量指针，反过来可以

```
int m=10,n=20,k=30;
const int * p1=&m,*p3=&n;
int * p2=&k;
p1 = p2;    //✓ 正确赋值情况1：常量指针=非常量指针
p1 = p3;    //✓ 正确赋值情况2：常量指针=常量指针
p2 = p1;    //✗ 错误赋值情况：非常量指针=常量指针
p2 = (int * ) p1;    //✓ 强制类型转换，正确!
```

- 函数参数为常量指针时，可避免函数内部不小心改变参数指针所指地方的内容

```
void MyPrintf( const char * p )
{    strcpy( p,"this");        //编译出错！
     printf("%s",p);           //✓
}
```

★C++新特性： const

3. const与引用　　(常引用)

- (1) 用法：定义引用时，前面加const关键字，即为常引用

```
int n;
const int & r = n;      //可以通过r引用n的值但不能修改n的值，n本身可以修改
r =5;                   //✗
n = 4;                  //✓
```

- const T & 和T & 是不同的类型!!!

  - ✓ const T & =T &
  - ✓ const T & =T
  - × T &=const T
  - × T &=const T &

  - T & 类型的引用或T类型的变量可以用来初始化const T & 类型的引用。
  - const T 类型的常变量和const T & 类型的引用则不能用来初始化T &类型的引用，除非进行强制类型转换。

★C++新特性：const

选择题：下面语法正确的一组是（ ）。



A
```
int n = 4;
int & r = n * 5;
```

C
```
int n = 8;
const int & r1 = n;
int & r2 = r1;
```

B
```
int n = 6;
const int & r = n;
r = 7;
```

D
```
int n = 8;
int & r1 = n;
const int r2 = r1;
```